Smart Constructors

onstructors · Money, Email, IBAN value objects with validated creation · Build with

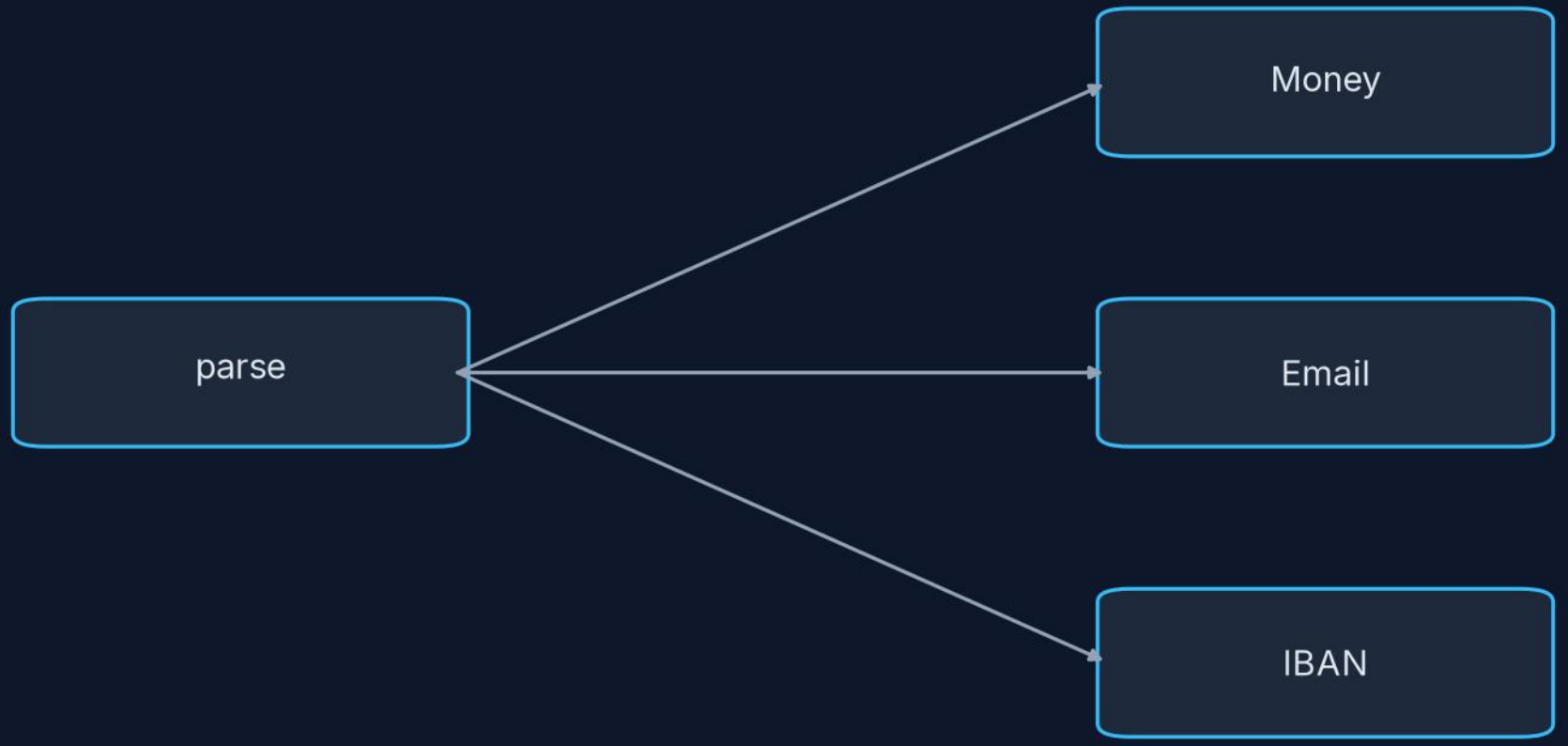
Today's Hook

- Problem: Money, Email, IBAN value objects with validated creation
- Pattern: Smart Constructors — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Money, Email, IBAN value objects with validated creation
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Smart Constructors: Money, Email, IBAN value objects with validated cr

Real-World Scenario

- Running example: Money, Email, IBAN value objects with validated creation
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.smartctor.problem;

/** Naive: public constructors / raw strings — invalid money/email/IBAN compile fine. */
public final class NaiveValues {
    public record RawMoney(String currency, int cents) {}
    public record RawEmail(String value) {}
    public record RawIban(String value) {}
    public static RawMoney money(String c, int cents) { return new RawMoney(c, cents); } // accepts -1
    public static RawEmail email(String v) { return new RawEmail(v); } // accepts "not-an-email"
    public static RawIban iban(String v) { return new RawIban(v); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.smartctor.problem;

import java.util.Optional;

/** Smart constructors: Money, Email, IBAN — invalid values cannot be constructed. */
public final class ValueObjects {
    private ValueObjects() {}

    public static final class Money {
        private final String currency;
        private final int cents;
        private Money(String currency, int cents) {
            this.currency = currency;
            this.cents = cents;
        }
        public String currency() { return currency; }
        public int cents() { return cents; }
        public static Optional<Money> of(String currency, int cents) {
            if (currency == null || currency.length() != 3) return Optional.empty();
            if (cents < 0) return Optional.empty();
            return Optional.of(new Money(currency.toUpperCase(), cents));
        }
```

Solution (2)

```
    public String toString() { return currency + ":" + cents; }
}

public static final class Email {
    private final String value;
    private Email(String value) { this.value = value; }
    public String value() { return value; }
    public static Optional<Email> of(String raw) {
        if (raw == null) return Optional.empty();
        String s = raw.trim();
        int at = s.indexOf('@');
        if (at <= 0 || at != s.lastIndexOf('@') || at == s.length() - 1) return Optional.empty();
        if (s.contains(" ")) return Optional.empty();
        return Optional.of(new Email(s.toLowerCase()));
    }
    public String toString() { return value; }
}

public static final class Iban {
    private final String value;
    private Iban(String value) { this.value = value; }
    public String value() { return value; }
    public static Optional<Iban> of(String raw) {
        if (raw == null) return Optional.empty();
        String s = raw.replace(" ", "").toUpperCase();
        if (s.length() < 15 || s.length() > 34) return Optional.empty();
        if (!s.matches("[A-Z]{2}[0-9]{2}[A-Z0-9]+")) return Optional.empty();
        return Optional.of(new Iban(s));
    }
```

Solution (3)

```
        public String toString() { return value; }
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.smartctor.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Smart Constructors: Money / Email / IBAN ===");
        System.out.println("--- naive allows illegal states ---");
        System.out.println(NaiveValues.money("US", -5));
        System.out.println(NaiveValues.email("not-an-email"));
        System.out.println(NaiveValues.iban("xx"));

        System.out.println("--- smart constructors ---");
        System.out.println("money=" + ValueObjects.Money.of("USD", 1999));
        System.out.println("badMoney=" + ValueObjects.Money.of("US", -1));
        System.out.println("email=" + ValueObjects.Email.of("A@B.C"));
        System.out.println("badEmail=" + ValueObjects.Email.of("nope"));
        System.out.println("iban=" + ValueObjects.Iban.of("DE89 3704 0044 0532 0130 00"));
        System.out.println("badIban=" + ValueObjects.Iban.of("12"));
    }
}
```

Walkthrough Output

```
=== Smart Constructors: Money / Email / IBAN ===
--- naive allows illegal states ---
RawMoney[currency=US, cents=-5]
RawEmail[value=not-an-email]
RawIban[value=xx]
--- smart constructors ---
money=Optional[USD:1999]
badMoney=Optional.empty
email=Optional[a@b.c]
badEmail=Optional.empty
iban=Optional[DE89370400440532013000]
badIban=Optional.empty
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Smart Constructors.

Related Patterns

- Contrast Smart Constructors with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Smart Constructors → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.